



Getting Started

Running & Extending the 4WD Torque-Vectoring Platform

A practical guide for a team member who is new to the project: from installing the tools and cloning the repository, through opening and running the simulation, to making changes and pushing them back. Assumes working MATLAB knowledge; explains the Git and Simulink-Project workflow from the ground up.

UNIVERSITY OF STRATHCLYDE MOTORSPORT

Companion to: the Handover Specification, the Implementation Tracker, and the Build Journal

What this platform is, in two minutes

This repository is a **modular Simulink simulation** for developing and comparing control strategies for a four-wheel-drive, hub-motor Formula Student car. You pick a controller and a torque-allocation strategy, pick a vehicle configuration (one motor, two, or four), run a standard manoeuvre, and read a set of metrics that tell you how well that combination performed. Every major block is swappable, so you can improve one part without breaking the others.

The whole system is one closed loop of referenced Simulink models, each its own file, joined by named signal contracts (Simulink buses). All numerical parameters live in one MATLAB file; the buses and parameters load automatically when you open the project. The diagram below is the signal flow you will see when you open the top model.

The one-paragraph mental model

Manoeuvre generates a steering input → **Reference** turns it into a target yaw rate → **Controller** compares target vs. actual and asks for a yaw moment → **Allocator** splits that moment into four wheel-torque requests → **Powertrain** turns requests into delivered torques → **Plant** (the vehicle model) computes the motion → the measured state feeds back to the Controller. **Metrics** read the logged signals afterwards.

You do not need to understand every block to be useful. The platform is built so that a newcomer can improve *one* block in a single term without needing to understand — or risk breaking — all the others. This guide gets you to that point.

The files you will actually touch

A handful of MATLAB files are the 'source of truth' and are easy to read and edit. The Simulink models (.slx) are binary — you edit them in Simulink, not a text editor.

- vehicleConfig.m — every parameter (geometry, mass, tyre, control gains) and the configuration selectors.

The single source of truth; change a number here, never inside a block.

- defineBuses.m — the signal contracts between blocks. Change this only when adding or removing a signal that crosses a block boundary.
- startup_project.m — runs automatically on project open; loads the buses and the config so everything is ready.
- referenceGen.m, doubleTrackModel.m, allocateBaseline.m, metrics.m — the maths inside the blocks (reference target, vehicle plant, torque split, performance metrics).
- YawTV_4WD.slx — the top model you run. It references the block models: Reference, Controller, Allocator, Powertrain, Plant, Manoeuvre.

1 Install the tools

You need three things: MATLAB (with Simulink), Git, and access to the private GitHub repository. Do these once per machine.

1.1 MATLAB + Simulink

Install **MATLAB R2025b** (the version the platform is developed and tested on) with the **Simulink** and **Stateflow** toolboxes. Strathclyde provides a campus licence — sign in with your university account at the MathWorks site, download the installer, and select MATLAB, Simulink and Stateflow at the product-selection step. Using the same R2025b avoids model-version warnings when you open the `.slx` files.

Why the exact version matters

Simulink models are saved in a version-specific binary format. Opening an R2025b model in an older MATLAB can fail or silently convert it. If you must use a different release, agree it with the team first — do not commit a model re-saved in another version without flagging it.

1.2 Git

Install **Git for Windows** from git-scm.com (accept the defaults; they are fine). Git is what version-controls the project — it records every change so nothing is ever lost and several people can work without overwriting each other. After installing, open a new MATLAB and confirm MATLAB can see Git: in the **Current Folder** panel, right-click → **Source Control** should be available. On Windows, Git's commit/merge hooks occasionally need the Git binary on MATLAB's library path — if MATLAB cannot find Git, restart MATLAB after installation so it picks up the new PATH.

1.3 GitHub access

The repository is **private**, so you need to be added as a collaborator and you need a way to authenticate. Ask the repository owner (currently N. Kostomoudis, GitHub user `Nikolaskost`) to add your GitHub account to the `usm-torque-vectoring` repository. Then create a **fine-grained Personal Access Token (PAT)** — this is the password Git will use over HTTPS:

- On GitHub: **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate new token**.
- Give it access to **only** the `usm-torque-vectoring` repository, with **Contents: Read and write** permission.
- Copy the token when shown (you cannot see it again). The first time you push, Git will ask for your username and password — paste the **token** as the password. Windows Credential Manager remembers it after that.

Email privacy — do this before your first push (avoids error GH007)

GitHub can reject a push that would publish your private email address. Prevent it by telling Git to use your GitHub `noreply` address. Find it on GitHub under **Settings** → **Emails** (it looks like `12345+username@users.noreply.github.com`), then run, in MATLAB's Command Window:

```
!git config --global user.name "Your Name"
!git config --global user.email "12345+username@users.noreply.github.com"
```

2 Clone the repository and open the project

‘Cloning’ means downloading your own local copy of the repository, linked to the GitHub original so you can pull updates and push changes.

2.1 Choose a location — NOT inside OneDrive

Put the repository somewhere simple and **outside any cloud-synced folder** (OneDrive, Dropbox, Google Drive). Cloud sync fights with Git over the same files and corrupts the repository. The convention on this project is `C:\Dev\usm-torque-vectoring`. Create the `C:\Dev` folder first.

2.2 Clone it

The easiest route is MATLAB's built-in clone. On the MATLAB **Home** tab: **New** → **Project** → **From Git**. Paste the repository URL (<https://github.com/NikolasKost/usm-torque-vectoring.git>), set the sandbox folder to `C:\Dev\usm-torque-vectoring`, and click **Retrieve**. Enter your GitHub username and your PAT (as the password) if prompted. MATLAB downloads the repository and opens it as a MATLAB Project in one step.

What a ‘MATLAB Project’ gives you

The repository contains a Project file (`.prj`). Opening the *Project* — rather than just the model — is what makes Simulink and Git cooperate: it sets the path, runs the startup file (loading buses + config automatically), and tracks file dependencies. Always open the project, not the bare `.slx`.

2.3 Confirm it is ready

When the project opens, `startup_project.m` runs automatically and loads the buses and the `cfg` structure into the workspace. Confirm in the Command Window:

```
cfg           % should print the parameter structure
cfg.actuator  % e.g. "RWD_single" | "RWD_twin" | "AWD_4"
```

If `cfg` is undefined, the startup script did not run — you probably opened the model instead of the project. Close everything, and open the `.prj` file.

3 Run a simulation and read the results

3.1 Open and run the top model

From the Project panel, open `YawTV_4WD.slx` — the top model. Press the green **Run** button, or run from the Command Window:

```
out = sim("YawTV_4WD");           % run the closed-loop simulation
out.logout.getElementNames       % see which signals were logged
```

The model simulates a step-steer manoeuvre and logs the reference and actual yaw rate. To see the tracking quality, run the metric harness:

```
M = metrics(out, cfg);           % RMSE / IAE / peak error etc.
disp(struct2table(M));
```

3.2 Change the configuration and re-run

The point of the platform is comparison. To switch which wheels are driven, change the actuator field and re-run — the same controller and allocator code runs unchanged, the `apply_topology` logic simply zeroes the undriven wheels:

```
cfg.actuator = "AWD_4";          % four-wheel drive
out = sim("YawTV_4WD");
M = metrics(out, cfg); disp(struct2table(M));
```

A habit worth forming on day one

Before you save a model or run any Git command, glance at the **Current Folder** bar and confirm it reads `usm-torque-vectoring`. MATLAB saves and runs Git relative to the current folder; if it has drifted elsewhere, files save to the wrong place and Git commands fail confusingly. A quick `cd('C:\Dev\usm-torque-vectoring')` is cheap insurance.

4 Making changes and sharing them (the Git workflow)

Git tracks the history of the project. The everyday loop is: **pull** the latest, make a change, **commit** it (a labelled snapshot), and **push** it back to GitHub so others get it. On this project we do commits and pushes through MATLAB's **Source Control** panel (the buttons), not typed Git commands — it stages files explicitly and avoids a class of mistakes.

4.1 Start every session by pulling

Before doing any work, get the latest version so your machine does not fall behind the other laptop and fork the history. In the Project tab, use **Source Control** → **Pull**. Do this first, every session.

4.2 Make your change

Edit a parameter in `vehicleConfig.m`, or a block in a model, or one of the maths files. Keep changes small and focused — one idea per commit is easier to review and to undo. Save model files from Simulink; if you edited the top model and it references unsaved sub-models, save with the `dirty-referenced-models` option so everything is written:

```
save_system("YawTV_4WD", "SaveDirtyReferencedModels", "on");
```

4.3 Commit (a labelled snapshot)

In the Project panel, files you changed show a modified marker. Open **Source Control** → **View and Commit Changes**, review the list, write a clear message describing *what* changed and *why*, and commit. Good messages read like Phase 3 step 16: pseudo-inverse allocator consuming per-wheel Fz. A new `.slx` file must be explicitly added before it can be committed — tick it in the commit dialog so it is not silently left out.

4.4 Push (share it)

Source Control → **Push** sends your commits to GitHub. If Push is refused with 'remote contains work you do not have', someone else pushed first — **Pull** to merge their changes in, then Push again. Because the model files are binary, two people editing the *same* model cannot auto-merge; coordinate so you are not editing the same `.slx` at once.

Trust the repository, not the panel icons

Occasionally the Project panel's file icons lag behind the true Git state. The ground truth is Git itself. From the Command Window these read-only checks never change anything and tell you exactly where you stand:

```
!git status           % what is modified / staged / ahead-behind
!git log --oneline -5  % recent commit history
!git show --stat HEAD  % files in the latest commit
```

5 Extending the platform (upgrading a block)

The platform is designed so a common upgrade touches **one** block. The rule is: a block can be replaced freely as long as it still honours its bus contract — the named signals in and out. Typical upgrades and what they touch:

- **New controller** (e.g. add a fuzzy or MPC controller): build it against the same Reference→Controller and Controller→Allocator buses; nothing else changes. Select it with one field in `cfg.arch.controller`.
- **Higher-fidelity plant or tyre**: edit the Plant block's maths; as long as the PlantBus outputs are unchanged, the controller above is unaffected.
- **New allocation strategy**: swap the Allocator block; the controller still emits the same demand and the motors still receive four torque requests.
- **New parameter**: add it to `vehicleConfig.m` only — never hard-code a value inside a block.

The golden rules (learned the hard way)

Keep these in mind whenever you edit the models

These are the things that have cost real time on this project. They are cheap to follow and expensive to ignore.

- **Confirm the current folder** reads `usm-torque-vectoring` before any `save_system` or Git action.
- **Changes to a model are only durable after a save**. Editing a block's code or a parameter from the command line lives in memory only — if the model closes before `save_system`, the change is lost. Save before closing.
- **Save the top model with** `SaveDirtyReferencedModels="on"` so referenced sub-models are written too.
- **New .slx files must be explicitly added** to Git, or they silently miss the commit.
- **Adding a feedback path around a MATLAB Function block** can break Simulink's signal-size inference; if a model that compiled suddenly will not, pin the block's input and output sizes explicitly rather than leaving them inherited.
- **Trust `git status` over the panel icons** when they disagree.

Keeping the documentation in step

Three documents travel with the code and should be kept current: the **Specification** (what the platform is and is meant to become), the **Implementation Tracker** (the live step-by-step checklist), and the **Build Journal** (the record of how each piece was built and why). Each is generated from a Python script in the repository, so updating one is a small edit to its script followed by a re-run. The lightweight habit is a one-line entry in `CHANGELOG.md` at every meaningful commit; the heavier milestone snapshots (re-generating the PDFs) happen only at version tags.

Where to go next

Read the **Specification** for the why, the **Implementation Tracker** for the what-next, and the **Build Journal** for how the current state was reached. Then pick the next unticked step in the tracker, or an upgrade from the list above, and work it one focused change at a time — pull first, commit with a clear message, push when it works.